



基于微服务架构的智能物流管理系统

汇报人：付苗
指导老师：牛志华
汇报时间：2026.03.27

目录

01 选题背景和意义

02 系统功能

03 系统架构和技术方案

04 当前进度

05 功能展示

06 项目亮点

07 后续计划

Part One

选题背景和意义



课题目标

本课题的目标是完成一套基于微服务的智能物流管理系统，系统覆盖顾客下单、商家发货、司机配送、顾客收货的完整业务流程，并在配送路线规划环节引入 AI，使系统能够结合多源数据自动生成更合理的配送路线。

选题背景

微服务架构实现灵活扩展与高效协同。就解耦与灵活性而言，微服务架构允许各模块独立开发、测试与部署，解决传统单体系统“牵一发而动全身”的维护难题。

大模型技术在数据理解、决策推理、多模态信息融合等方面展现出显著优势，通过引入大模型，可对订单需求、历史路况、天气信息等多源数据进行综合分析，辅助完成需求预测、多方案调度决策生成，提高物流系统的整体智能化水平和响应速度。

Part Two

系统功能



管理员 Admin

- 检测系统整体情况
- 管理所有用户账号
- 监控全部订单状态
- 管理仓库信息

商家 Shop

- 管理店铺信息与商品
- 管理仓库与库存
- 处理订单发货
- 查看店铺营收数据（后续开发）



顾客 Customer

- 浏览商城商品、加入购物车
- 下单、支付、查看订单进度
- 查看配送路径
- 管理收货地址

司机 Driver

- 接收配送任务
- 实时上报 GPS 位置
- 查看系统规划路线
- 管理绑定车辆信息

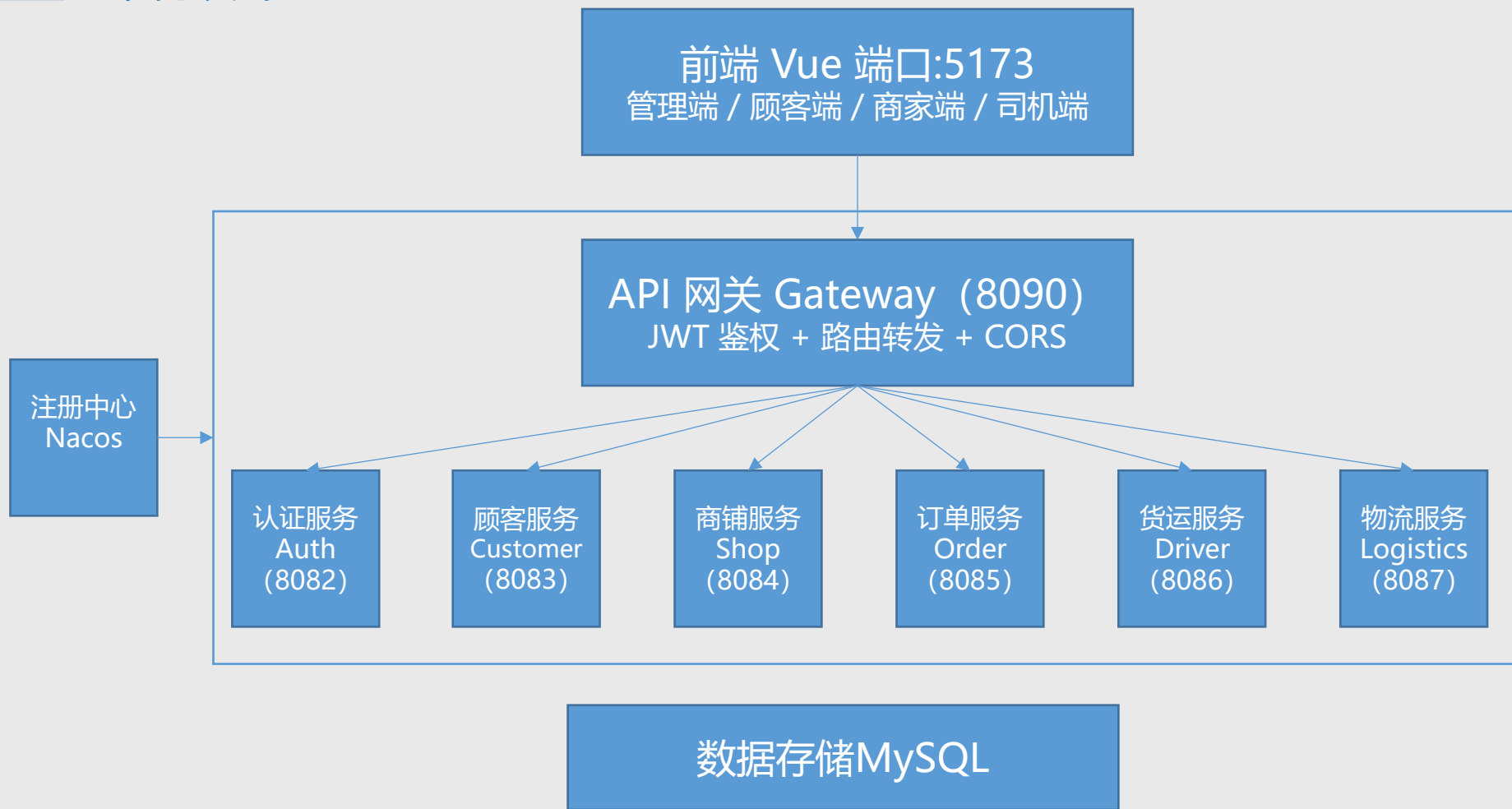
Part Three

系统架构和技术方案



03

Part Three 架构设计



当前服务间通信使用OpenFeign，预计 后续扩展为RabbitMQ，并组合使用Redis做系统缓存。

03

Part Three 技术方案

后端

技术	用途
Spring Boot	服务基础框架
Spring Cloud	微服务治理
Spring Cloud Alibaba	Nacos集成
MyBatis Plus	ORM数据访问
OpenFeign	服务间调用
GraphHopper	路图信息读取
SpringDoc OpenAPI	接口文档
JWT	JWT令牌

前端

技术	用途
Vue3 + Vite8	前端框架
TypeScript	类型安全
Element Plus	UI组件
Pinia	状态管理
Axios	请求处理
Leaflet	地图渲染
Vue Router	路由管理

Part Four

当前进度



04

Part Four

当前进度-整体进度

模块	实现功能	状态
API 网关	JWT令牌, 角色权限, 路由转发	✓ 完成
认证服务 (auth)	用户登录, Token 签发	✓ 完成
用户管理 (user)	管理员账号管理	✓ 完成
顾客服务 (customer)	顾客信息, 地址管理	✓ 完成
商铺服务 (shop)	商品管理, 仓库、库存管理	✓ 完成
订单服务 (order)	订单状态管理、商品快照	✓ 完成
司机服务 (driver)	司机信息、车辆、接单任务	✓ 完成
物流服务 (logistics)	三种路线策略、轨迹追踪	✓ 完成
前端 (Vue 3)	4个角色页面、25 个子页面	✓ 完成
容器化部署	Docker+Kubernetes	🔧 待完成
消息队列	RabbitMQ消息队列	🔧 待完成
缓存层	Redis缓存	🔧 待完成

整体进度

后端：全部模块主干功能已完成

前端：四个角色的核心功能页面均已覆盖

模块	实现功能	状态
认证服务 (auth)	用户注册	 待完成
用户管理 (user)	管理员审核商铺、司机注册信息	 待完成
订单服务 (order)	订单评价、状态变更管理	 待完成
物流服务 (logistics)	增加除历史信息外的信息源供大模型决策	 待完成
前端 (Vue 3)	上述功能对应页面开发	 待完成

Part Five

功能展示



Part Six

项目亮点



05

Part Five

项目亮点-整体背景

GraphHopper + OpenStreetMap

路网数据来源于 OpenStreetMap，用 GraphHopper 加载解析后构建路网图缓存到 graph-cache` 目录。路网图里每个节点代表一个路口，每条边代表一段道路，边上带有距离属性。

坐标映射

在运行 A* 之前，需要先把用户输入的经纬度坐标映射到路网上最近的节点，这一步通过 GraphHopper 的 `LocationIndex.findClosest()` 实现——它会在路网中找到距离给定坐标最近的有效路网节点，确保起终点都在真实道路上

```
// — 第一步：经纬度 → 最近路网节点 —————  
Snap startSnap = locationIndex.findClosest(startLat, startLon, EdgeFilter.ALL_EDGES);  
Snap endSnap   = locationIndex.findClosest(endLat,   endLon,   EdgeFilter.ALL_EDGES);
```



ADD TEXT



ADD TEXT



ADD TEXT

核心公式:

$$f(n) = g(n) + h(n)$$

$g(n)$ 是从起点沿路网走到当前节点 n 的实际代价, 用已走过的路网边距离累加来表示。

$h(n)$ 是当前节点到终点的启发式估计, 当前用的是 Haversine 球面距离公式, 也就是地球表面两点的球面直线距离。

加权A*

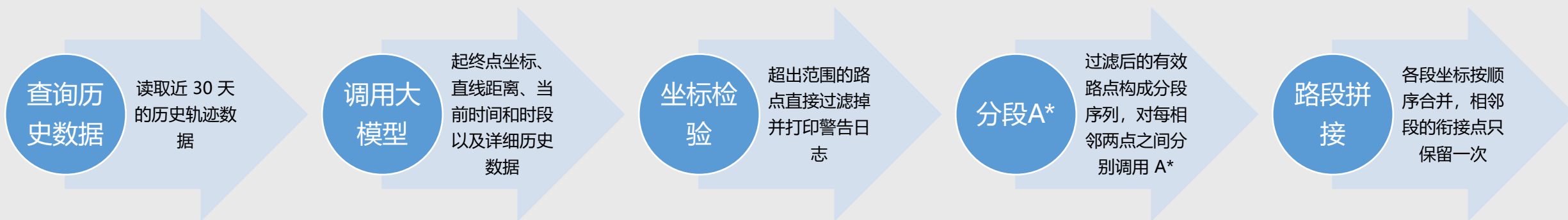
在标准 A* 的基础上, 引入启发函数权重 ε : $f(n) = g(n) + \varepsilon \times h(n)$

当 $\varepsilon = 1.0$ 时, 就是标准最优 A*; 当 $\varepsilon > 1.0$ 时, 启发函数被"膨胀", 算法更激进地趋向终点, 会探索出与最优路线不同的备选路径。

05

Part Five

项目亮点-LLM_WAYPOINT

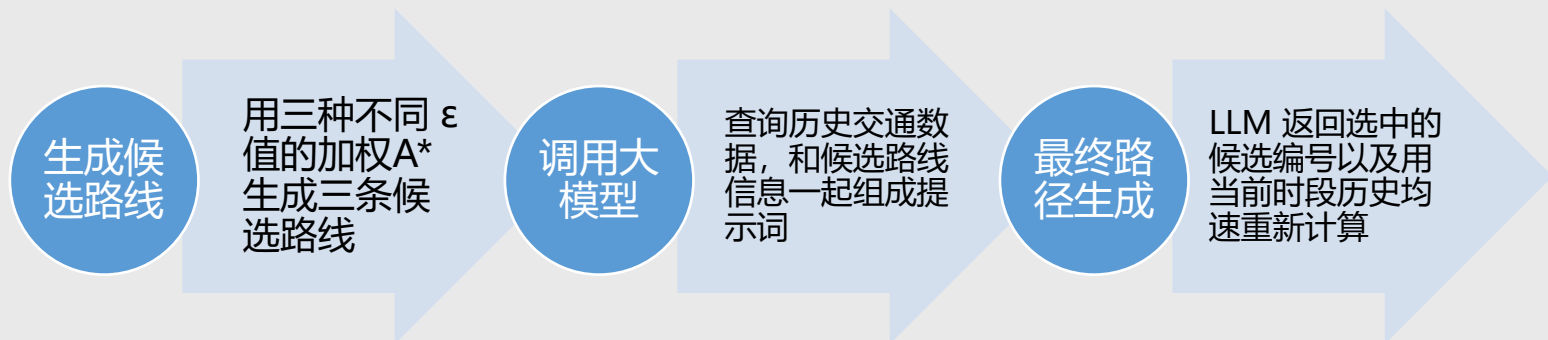


降级机制

如果 LLM 调用失败（网络超时等），或者过滤后没有有效路段，系统会自动降级为纯 A* 直达规划，保证路线一定能生成，用户不会看到报错。

05

Part Five 项目亮点-LLM_JUDGE



策略对比

LLM_WAYPOINT 中 LLM 决定“走哪个方向”，A* 在真实路网上执行每段规划，如果出现 LLM 幻觉坐标在一定程度上会对路线整体质量产生影响；LLM_JUDGE 中 LLM 参与的是评估层，所有候选路线都已经是合法的路网路线，LLM 只是做综合评估选最优的。

当前默认执行 LLM_WAYPOINT 模式

Part Seven

后续计划





容器化部署

后续计划使用 Docker 进行容器化部署。计划为每个服务编写 Dockerfile，并通过 Kubernetes 完成容器编排与服务管理。前端使用 Nginx 容器进行托管，并配置反向代理到后端网关。这样整个系统可以通过 Kubernetes 统一部署和管理，具备良好的扩展性与可迁移性。



消息队列

目前服务间都是 Feign 同步调用，一旦某个服务响应慢或挂掉，整个调用链都会阻塞。引入 RabbitMQ 之后，例如发货这个操作，order-service 只需要发一条“订单发货”消息，logistics-service 订阅到之后异步去生成路线，两个服务就解耦了。



Redis 缓存

商品列表这类数据读多写少，现在每次都查数据库，压力比较大。加了 Redis 之后，第一次查完放进缓存，后续直接从缓存取，能大幅降低数据库压力。另外 Token 黑名单（用户退出登录后让 Token 失效）也计划用 Redis 来实现。



谢谢观看

汇报人：付苗

汇报时间：2026.03.27